

CVA lattice insertion, explained

Accompanies `Lattice.scala` and the loop-based `LatticeSimple.scala`.

Accurate short description. An incoming instance becomes a temporary singleton CVA target. A bounded search starts at specific bottom nodes and moves upward through more-general parents. It chooses a locally compatible generator. Existing locally found nodes that are mutually CVA-equivalent to a target are reused; otherwise the target is materialized. Intermediate child-union concepts may be built. Edges are ordered by strict symbolic extent inclusion. Finally, the incoming instance is added to the generator and its bounded ancestors, whose CVA models are refitted.

1. What a node represents

Every node contains a symbolic extent (symbols), a vector extent, and a CVA model consisting of a common vector, projection, and radius. A node classifies a vector when its projected residual distance is at most its learned radius.

```
node.classifies(x) ⇔ CVA distance(node.model, x) ≤ node.model.radius
```

A parent is *more general* and has a strict superset of its child's symbols. A child is more specific. Therefore bottom nodes—nodes without children—are the most-specific current concepts.

2. Main insertion flow

```
singleton target
↓
sample closest bottom nodes
↓ search through parents
compatible generator
↓
loop over bounded child-union work
↓
reuse equivalent node or materialize target
↓
repair local symbolic cover edges
↓
add instance to generator + ancestors and refit
```

Generator search

`maximalConcept` starts from at most eight close bottom nodes. A priority queue visits closer nodes first. If a node is incompatible, its parents are queued. If compatible, that path stops because its parents are less specific. The smallest compatible extent is selected. At most 24 distinct nodes are visited, so the algorithm is deliberately local rather than exhaustive.

Compatibility combines two learned conditions:

1. `intentSubset(concept, target)`: the concept classifier accepts the required fraction of the target extent. For a singleton and coverage 1.0, it accepts the incoming instance.
2. `coherentWithIncoming`: the concept is close to the incoming vector, and the singleton model is close to every existing member of the concept.

Equivalence

Equivalence is mutual learned coverage, not equality of symbol sets:

$$A \approx B \Leftrightarrow \text{coverage}(A, B.\text{extent}) \geq \text{threshold} \\ \wedge \text{coverage}(B, A.\text{extent}) \geq \text{threshold}$$

Optional configuration also requires each node to classify the other's common vector. The check is local: it checks the generator and the result of another bounded search during materialization, not every lattice node.

Topology and final update

Learned classification is not guaranteed to be transitive, so it is not trusted for graph direction. Edges use strict symbolic inclusion: `parent.symbols  $\supset$  child.symbols`. During planning, accepted ancestors are viewed as if they already contained the new symbol. Only after structural work finishes is the incoming item inserted into the generator and its bounded ancestor closure; each changed node is then refitted.

3. What the original recursion does

The recursion is not the upward lattice search; `maximalConcept` performs that search with a loop. Recursive insertion explores child-union candidates:

1. Select a generator for the current target.
2. If the generator is equivalent, return it.
3. Otherwise inspect up to four nearest generator children.
4. Reuse a child that contains the target; discard an incoherent child.
5. For a coherent non-containing child, create a target fitted from `child.extent  $\cup$  missing target instances`.
6. Insert that union target one level deeper and use its returned canonical node as a prospective parent.
7. After all child results return, materialize the current target and repair local edges.

The `activeCalls` set guards the pair (`target symbols`, `generator ID`), preventing a repeated work path from cycling.

4. How LatticeSimple replaces recursion

`LatticeSimple` extends `Lattice` and overrides only `addIntentRecursively`. It stores unfinished calls as `Frame` objects in an `ArrayDeque`. A frame records:

- current target, preferred generator, and depth;

- selected generator and recursion-guard key;
- ordered children and the next-child index;
- whether it is waiting for a deeper result;
- the reduced prospective-parent list.

```

push initial frame
while frames are non-empty:
    frame = top
    if a deeper frame returned:
        accept its canonical node as a candidate
    else if frame is new:
        select generator; finish early or initialize child work
    else if an unprocessed child exists:
        accept, reject, or push a child-union frame
    else:
        materialize target, repair edges, pop frame, return result upward

```

This is the same post-order dependency as recursion: the parent frame pauses, the nested union finishes, and only then does the parent consume its result. The JVM call stack remains constant; the heap-based deque holds the state explicitly.

5. What happens at depth 2 or more?

Setting	Behavior	Trade-off
maxDepth = 0	No generator children are inspected.	Simplest and cheapest; creates no child-union intermediates.
maxDepth = 1	The initial frame inspects children and may push one union level. Depth-1 frames do not inspect their children.	Matches the original class's current bound.
maxDepth = 2	A depth-1 union frame may inspect its own generator's children and create depth-2 unions.	Can discover a more refined local chain, with more fitting and searches.
maxDepth = D	Child expansion occurs at depths 0..D-1; frames at depth D finalize without child expansion.	Potential work grows roughly with branching; guards and other fixed search limits still apply.

With up to four children per frame, the theoretical frame count through depth D resembles $1 + 4 + 4^2 + \dots + 4^D$. Actual work is normally smaller because children may be accepted, rejected, deduplicated by target cache, or stopped by the active-call guard. Each frame can also run bounded generator searches and fit a new CVA target, so increasing depth can be materially more expensive.

Important: greater depth improves local exploration, not global completeness. Bottom sampling, eight search starts, 24 visits, and four children per frame remain bounded. A deeper search can also make insertion-order and learned-equivalence effects more visible; validate the lattice and benchmark representative data before choosing a large value.

6. Usage

```
val lattice = new LatticeSimple(  
    cva = CVA(dimension),  
    config = LatticeConfig(),  
    maxDepth = 2,  
    maxChildrenPerFrame = 4  
)  
  
val objectConcept = lattice.addIntent(instance)  
lattice.validate()
```

Use depth 1 for behavior corresponding to the original implementation. Use depth 0 for the clearest minimal insertion. Try depth 2 only when the additional intermediate concepts improve results enough to justify the extra CVA fits and local searches.